

ProteinMPNN: What It Can Do

Undergraduate-friendly high-level summary based on the ProteinMPNN paper, official GitHub, and provided command-line examples

Plain-language explanation: ProteinMPNN is like structure-aware autocomplete for proteins. You give it a 3D backbone shape and optional rules, and it proposes amino-acid sequences that should be compatible with that shape.

1. One-sentence overview

ProteinMPNN is a deep-learning protein sequence design model. Its core job is to propose amino-acid sequences for a given protein backbone structure.

Plain-language explanation: The model does not start from a blank page. It starts from a backbone: the 3D path of the protein chain. Then it fills in amino acids that make sense for that 3D environment.

2. The design problem it solves

Protein sequence design asks: given a desired protein backbone, what amino-acid sequence is compatible with that backbone? ProteinMPNN treats this as a probability problem: for each position, it estimates which amino acids are likely given the 3D neighborhood around that position.

Plain-language explanation: Imagine the backbone as a fixed wire sculpture. ProteinMPNN chooses which amino-acid beads should go at each location so that the final protein sequence is plausible for that sculpture.

3. What ProteinMPNN uses as input

- A protein structure file containing backbone atom coordinates.
- A choice of which chain or chains should be designed.
- Optional fixed positions that must keep their original amino acids.
- Optional design-only positions, where only a small selected region is allowed to change.
- Optional tied positions, where two or more positions must receive the same amino acid.
- Optional amino-acid bias, which makes some amino acids more or less likely.
- Optional PSSM-style prior information, which can guide the model toward known or preferred residue patterns.
- Optional input sequence in FASTA format when the goal is to score an existing sequence instead of generating a new one.

Plain-language explanation: The input is not just a sequence. The most important input is the 3D backbone. The extra files are rule sheets: they tell the model what it is allowed to change, what must stay fixed, and whether some amino acids should be preferred or avoided.

4. What ProteinMPNN returns as output

- Candidate amino-acid sequences, usually written in FASTA-like output files.
- A score for each generated sequence. Lower negative log-probability generally means the sequence is more compatible with the model probability distribution.
- A global score that can include both designed and fixed residues.
- Metadata such as designed chains, fixed chains, temperature, model name, and seed.
- Optional residue-level probability tables that show what amino acids the model prefers at each position.

Plain-language explanation: The output is a ranked list of possible protein sequences. It is not a final proof that a sequence will work experimentally. It is a fast way to generate and prioritize candidates.

5. Main things ProteinMPNN can do

Capability	What it means technically	More understandable explanation
------------	---------------------------	---------------------------------

Design a monomer sequence	Given a single-chain backbone, ProteinMPNN can generate sequences for that chain.	Use this when the whole protein chain can be redesigned.
Design multi-chain structures	ProteinMPNN can design selected chains while using the other chains as fixed structural context.	Use this when the structure contains more than one chain, but only some chains should change.
Run directly on one structure file	A single structure file can be passed directly to the main run script without first preparing a large folder of parsed structures.	Good for quick tests on one structure.
Generate many sequences per target	The same backbone can be sampled multiple times to produce many candidate sequences.	This is useful because there is usually not one single possible answer; many sequences may fit the same shape.
Control diversity with sampling temperature	Lower temperature gives more conservative, high-probability sequences. Higher temperature gives more diverse sequences.	Temperature is like creativity level. Low values stay close to the model favorite choices; higher values explore more alternatives.
Fix selected residues	Some positions can be locked so their amino acids do not change during design.	This is useful when certain residues are known to be important and should be preserved.
Design only selected residues	The opposite mode is also possible: only a specified set of residues can change and everything else is held fixed.	This is useful for local redesign, such as changing only a short region instead of the whole protein.
Tie positions together	Two or more positions can be forced to receive the same amino acid.	This is useful when repeated or matched positions should stay identical.
Design homooligomers	For structures with repeated identical chains, equivalent positions can be tied so the chains receive matching sequences.	This keeps repeated chains consistent rather than letting each copy drift independently.
Add amino-acid bias	The model can be nudged to prefer or avoid certain amino acids globally or by position.	This is like giving the model a preference list before it chooses residues.
Use PSSM-style guidance	External residue-preference information can be blended with model probabilities.	This lets prior sequence information influence the design instead of relying only on the model.
Score existing sequences	ProteinMPNN can score a given sequence on a given backbone without generating a new sequence.	This is useful for comparing variants and asking which sequence looks more compatible with the same backbone.
Return probability profiles	ProteinMPNN can output amino-acid probabilities at each position.	This shows what the model thinks each position wants, even before choosing a final sequence.

6. What the provided command-line examples show

The provided scripts are practical examples of the modes above. They show how the same main program can be run with different input flags to perform different design or scoring tasks.

Script example	Capability demonstrated
example_1	Design sequences for single-chain structures.
example_2	Design selected chains in multi-chain structures.
example_3	Run design directly from one structure file.

example_3_score_only	Score a structure-sequence pair without generating a new design.
example_3_score_only_from_fasta	Score sequences supplied from a FASTA file on a structure.
example_4	Keep selected positions fixed during design.
example_4_non_fixed	Design only selected positions and keep the rest fixed.
example_5	Combine fixed positions with tied positions.
example_6	Tie equivalent positions across repeated chains.
example_7	Return amino-acid probability profiles instead of final designs.
example_8	Add a global amino-acid bias.
example_pssm	Blend model probabilities with PSSM-style residue preferences.
custom local-design script	Design only positions 1-9 in one chosen chain and sample 3000 candidate sequences.

Plain-language explanation: The scripts differ mostly in their flags. The same model can act as a full-sequence designer, a local mutagenesis designer, a sequence scorer, or a probability-profile generator depending on which flags are used.

7. How constraints control the design

ProteinMPNN is flexible because the user can define rules before sampling sequences. These rules do not change the learned model itself; they change which positions are sampled and how the sampling probabilities are adjusted.

- Chain selection decides which chain or chains are redesigned.
- Fixed positions protect residues that should not mutate.
- Design-only positions restrict redesign to a small local region.
- Tied positions enforce identical amino-acid choices across matched positions.
- Amino-acid bias changes the probability of selected residue types.
- PSSM-style guidance mixes prior residue preferences with the model distribution.

Plain-language explanation: Think of constraints as instructions to the model: redesign this part, do not touch that part, make these positions match, and prefer or avoid certain residue types.

8. What ProteinMPNN is good at

- Rapidly generating many plausible sequences for a backbone.
- Handling both single-chain and multi-chain structures.
- Performing local redesign while keeping most of a protein fixed.
- Producing sequence scores that help prioritize candidates.
- Producing residue-level probability profiles that help identify flexible or constrained positions.
- Supporting symmetry-like constraints through tied positions.

Plain-language explanation: ProteinMPNN is especially useful when you already have a 3D backbone and want many reasonable sequence options quickly.

9. Important limitations

- ProteinMPNN does not prove that a generated sequence will fold, express, or function in a real experiment.
- The score is a model probability score, not a direct measurement of activity.
- The model follows the backbone and user constraints; it does not automatically know every biological objective.
- A sequence that looks good to ProteinMPNN may still need additional computational review and experimental testing.
- The quality of the input backbone and the chosen constraints strongly affect the usefulness of the output.

Plain-language explanation: The model is a candidate generator and scorer. It helps narrow the search space, but it does not replace validation.

10. Minimal coding mental model

1. Prepare one or more protein backbone structure files.
2. Decide which chains and residues should be designed.
3. Prepare optional rule files for fixed positions, tied positions, amino-acid bias, or PSSM-style guidance.
4. Run the main ProteinMPNN script with the desired flags.
5. Collect the generated sequences, scores, and optional probability outputs.
6. Filter or rank candidates according to the design goal.

Plain-language explanation: From a coding viewpoint, ProteinMPNN is a command-line program with structured inputs and structured outputs. Most practical use is about preparing the right input files and choosing the right flags.

11. Quick reference table

Goal	Mode to use
Redesign a whole single chain	Monomer design
Redesign selected chains in a complex	Multi-chain design with chain selection
Mutate only a local region	Design-only positions
Protect important residues	Fixed positions
Keep repeated positions identical	Tied positions
Prefer or avoid residue classes	Amino-acid bias
Use prior residue information	PSSM-style guidance
Compare existing sequences	Score-only mode
Inspect model preferences per position	Probability-only mode

12. References used

- Dauparas J. et al. Robust deep learning-based protein sequence design using ProteinMPNN. Science, 2022. DOI: 10.1126/science.add2187.
- Official ProteinMPNN GitHub repository by dauparas/ProteinMPNN. README and example scripts accessed for command-line modes and input flags.
- Provided command-line example scripts: example_1 through example_8, score-only examples, PSSM example, and custom local-design script.